

Team 4 — Frontend (React/Vite) Production Roadmap

Real-time Progress UI · API Integration · PDF Export · Mobile Responsive

Frontend Team

YourCrawl Dark Pattern Auditor

Generated: 16 August 2026

Frontend (React/Vite) Production Roadmap

Assigned To: Frontend Team Lead

Priority: HIGH — The product's face. First impression for every user.

What Exists Today (Current State Assessment)

Component	File/Location	Status
-----------	---------------	--------

React + Vite setup	<code>vite.config.js</code> , <code>package.json</code>	Functional
Routing	React Router DOM	Working
Landing Page	<code>src/features/landing/</code>	Present
Dashboard	<code>src/features/dashboard/</code>	Present
Audit History	<code>src/features/audits/</code>	Present
Analysis View	<code>src/features/analysis/</code>	Present
Report View	<code>src/features/report/</code>	Present
Roadmap View	<code>src/features/roadmap/</code>	Present
Compliance View	<code>src/features/compliance/</code>	Present
AI Assistant	<code>src/features/aiAssistant/</code>	Present
Auth Views	<code>src/features/auth/</code>	Present
API Service	<code>src/services/api.js</code>	Present (927 bytes — very thin)
State Management	Unknown	Feature-level state only
Real-time Progress UI	None	MISSING
SSE / WebSocket Client	None	MISSING
Export (PDF/CSV)	None	MISSING
Skeleton Loading	None	MISSING
Mock Data Usage	<code>mockData.js</code> referenced	STILL USED IN PRODUCTION VIEWS
Mobile Responsiveness	Unknown	UNVERIFIED
Error Boundary	None seen	MISSING

Critical Flaws Found

1. No Real-Time Audit Progress UI (Showstopper for UX)

The Backend currently runs audits synchronously and blocks for 2-3 minutes. Once the Backend implements the Job Queue (Team 1 Feature 1), the Frontend must display real-time progress. Without this:

- Users stare at a blank screen or spinner
- Users refresh the page and lose context
- Users think the product is broken

This is the single most important Frontend feature to build.

2. Mock Data Still Used in Production Views

File: `src/features/` — multiple components reference `mockData.js` The Frontend was built with mock data for development. Several views are wired to static mock data rather than the real API. Shipping with mock data means:

- Users see fake audit data, not their actual results
 - Audit history shows fake entries
 - Compliance scores are hardcoded
-

3. The `api.js` Service Is Only 927 Bytes

File: `src/services/api.js` This file is extremely thin (less than 1KB). It cannot contain all the API calls needed for:

- Auth (login, logout, register, OAuth)
- Crawl (discover routes, start crawl, poll status)
- Audit history (list, get by ID)
- RAG AI Assistant (query, query-with-audit)
- User profile (get, update)
- Usage stats

Most of these calls are likely scattered across components using ad-hoc `fetch()` calls, making the codebase inconsistent and unmaintainable.

4. No Skeleton Loading States

When API calls are in flight, components either show nothing or a plain spinner. Production-quality apps show skeleton screens (placeholder shapes that match the layout) to reduce perceived wait time and prevent layout shift.

5. No Error Boundary or Global Error Handling

If a React component throws an uncaught error, the entire app crashes with a white screen. There is no error boundary wrapping the route tree to provide graceful degradation.

6. No Export Functionality

Enterprise users need to share audit results with their legal team, developers, or management. Currently there is no way to export a report as PDF, CSV, or share a public link.

7. AI Assistant (aiAssistant) Is Likely Not Wired to RAG Service

The `aiAssistant` feature exists in the Frontend, but given the thin `api.js` and the RAG service being complex, it is likely showing static or mock responses rather than real Gemini + legal RAG answers.

8. No Mobile Responsiveness Verification

The Frontend uses Tailwind CSS which can be mobile-first, but without explicit verification, complex data-heavy views (audit tables, compliance charts, roadmap boards) may be broken on mobile.

User Journey: Current vs Target State

Current State (Broken UX):

```
User enters URL → Clicks "Start Audit"
→ [2-3 minute freeze with no feedback]
→ Either timeout error OR sudden page update
→ User confused about what happened
```

Target State (Production Quality):

```
User enters URL → Clicks "Start Audit"
→ [Route Discovery Modal] – "Found 12 pages, select which to scan"
→ Clicks "Start Audit"
→ [Real-time Progress Bar]
  "Crawling homepage... (1/3 pages)"
  "Running ML analysis... 45%"
  "Generating AI report... Almost done!"
→ [Report Dashboard slides in with animation]
→ User can ask AI Assistant: "Which finding is most urgent?"
→ User clicks "Export PDF" to share with their team
```

Component Architecture: Target State

```
src/
├─ services/
|   ├─ api.js           <- Thin Axios/fetch wrapper with base URL + auth token
|   ├─ auditApi.js      <- All audit-related API calls
|   ├─ authApi.js       <- Auth API calls
|   ├─ ragApi.js        <- RAG / AI Assistant API calls
|   └─ userApi.js       <- User profile + usage API calls
|
├─ hooks/
|   ├─ useAuditStream.js <- SSE connection to audit progress stream
|   ├─ useAuditHistory.js <- Fetches and caches audit history
|   ├─ useRagChat.js     <- RAG chat state management
|   └─ useAuth.js        <- Auth state + token management
|
├─ stores/ (if using Zustand)
|   ├─ auditStore.js     <- Current audit job state
|   └─ authStore.js      <- User authentication state
|
├─ components/
|   ├─ common/
|   |   ├─ ErrorBoundary.jsx
|   |   ├─ SkeletonCard.jsx
|   |   ├─ SkeletonTable.jsx
|   |   ├─ ProgressBar.jsx
|   |   └─ EmptyState.jsx
|   └─ layout/
|       ├─ Sidebar.jsx
|       ├─ TopNav.jsx
|       └─ AppLayout.jsx
|
└─ features/
    ├─ audit/           <- URL input + route selection + audit trigger
    ├─ auditProgress/   <- Real-time progress (NEW)
    ├─ report/          <- Full audit report view
    ├─ aiAssistant/     <- Wired to RAG service (currently broken)
    ├─ compliance/      <- Compliance scores per framework
    ├─ roadmap/         <- Remediation roadmap kanban
    └─ export/          <- PDF / CSV export (NEW)
```

Feature Tasks to Build

Feature 1 — Real-Time Audit Progress UI (HIGHEST PRIORITY)

Why: This is the single most impactful UX improvement possible. Once the Backend has SSE, the Frontend must consume it. **Implementation Steps:**

13. Create `src/hooks/useAuditStream.js`:

```
export function useAuditStream(jobId, onComplete, onError) {  
  const [progress, setProgress] = useState(0);  
  const [stage, setStage] = useState('Initializing...');  
  
  useEffect(() => {  
    if (!jobId) return;  
  
    const eventSource = new EventSource(`${api/crawl/status/stream?jobId=${jobId}`, {  
      withCredentials: true  
    });  
  
    eventSource.onmessage = (e) => {  
      const data = JSON.parse(e.data);  
      if (data.progress) setProgress(data.progress);  
      if (data.stage) setStage(data.stage);  
      if (data.status === 'completed') {  
        onComplete(data.reportId);  
        eventSource.close();  
      }  
      if (data.status === 'failed') {  
        onError(data.error);  
        eventSource.close();  
      }  
    };  
  
    return () => eventSource.close();  
  }, [jobId]);  
  
  return { progress, stage };  
}
```

14. Create `src/features/auditProgress/AuditProgressModal.jsx`:

```

export function AuditProgressModal({ jobId, url, onComplete }) {
  const { progress, stage } = useAuditStream(jobId, onComplete, handleError);

  return (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
      <div className="bg-white rounded-2xl p-8 w-full max-w-md shadow-2xl">
        <h2>Auditing {url}</h2>
        <div className="mt-4 h-3 bg-gray-100 rounded-full overflow-hidden">
          <div
            className="h-full bg-gradient-to-r from-blue-500 to-violet-500 transition-all duration-500"
            style={{ width: `${progress}%` }}
          />
        </div>
        <p className="mt-3 text-sm text-gray-600">{stage}</p>
        <p className="text-right text-sm font-medium text-blue-600">{progress}%</p>
      </div>
    </div>
  );
}

```

15. Wire the progress modal into the audit trigger flow in `src/features/audit/`.

Acceptance Criteria: After clicking "Start Audit", a progress modal appears immediately and updates in real-time. The modal closes and the report loads automatically when done.

Feature 2 — Replace All Mock Data with Real API Calls

Implementation Steps:

16. Audit every component for `import mockData` or hardcoded data arrays. Remove all of them.

17. Create a centralized API service layer:

`src/services/auditApi.js`:

```
import api from './api.js';

export const startAudit = (url, routes, reportId) =>
  api.post('/api/crawl', { url, routes, reportId });

export const discoverRoutes = (url) =>
  api.post('/api/crawl/discover', { url });

export const getAuditHistory = () =>
  api.get('/api/crawl/history');

export const getAuditById = (id) =>
  api.get('/api/crawl/history/${id}');
```

`src/services/ragApi.js`:

```
import api from './api.js';

export const queryRag = (question, auditData, sessionId) =>
  api.post('/api/rag/query', { question, auditData, sessionId });
```

18. Update all components to use these API functions instead of mock data.

19. Add loading and error states to every component that fetches data.

Acceptance Criteria: Removing `mockData.js` from the project causes zero runtime errors.

Feature 3 — Skeleton Loading States

Implementation Steps:

20. Create `src/components/common/SkeletonCard.jsx`:

```
export function SkeletonCard() {
  return (
    <div className="animate-pulse bg-white rounded-xl p-6 border border-gray-100">
      <div className="h-4 bg-gray-200 rounded w-1/3 mb-3" />
      <div className="h-8 bg-gray-200 rounded w-1/2 mb-2" />
      <div className="h-3 bg-gray-100 rounded w-2/3" />
    </div>
  );
}
```

21. Create `src/components/common/SkeletonTable.jsx` for the audits history table.

22. In every component that fetches data:


```

if (isLoading) return <SkeletonCard />;

if (error) return <ErrorState message={error} />;

return <ActualComponent data={data} />;

```

Acceptance Criteria: Every data-fetching view shows a skeleton during loading. No blank screens or layout flashes.

Feature 4 — Route Discovery Modal (Multi-Page Audit)

Why: The Backend supports multi-page audits (`POST /api/crawl/discover` exists) but the Frontend likely does not expose this to users. **Implementation Steps:**

23. Create `src/features/audit/RouteDiscoveryModal.jsx`:

```

export function RouteDiscoveryModal({ baseUrl, routes, onConfirm, onCancel }) {

  const [selected, setSelected] = useState(new Set([''])); // base URL always selected

  return (

    <div className="modal">

      <h2>Select pages to audit</h2>

      <p>Found {routes.length} internal pages for {baseUrl}</p>

      {routes.map(route => (

        <label key={route.path} className="flex items-center gap-3 p-3 hover:bg-gray-50 rounded-lg cursor-
pointer">

          <input

            type="checkbox"

            checked={selected.has(route.path)}

            onChange={() => toggleRoute(route.path)}

          />

          <span>{route.label}</span>

          <span className="text-gray-400 text-sm">{route.path}</span>

        </label>

      ))}

      <button onClick={() => onConfirm(Array.from(selected).map(p => routes.find(r => r.path ===
p)?.fullUrl).filter(Boolean))}>

        Audit {selected.size} pages

      </button>

    </div>

  );
}

```

Acceptance Criteria: After entering a URL, users can see all discovered pages and choose which ones to include in the audit.

Feature 5 — AI Assistant Wired to RAG Service

Why: The AI Assistant feature exists but is likely returning mock/static responses. **Implementation Steps:**

24. Implement `src/hooks/useRagChat.js`:

```
export function useRagChat(auditData) {  
  const [messages, setMessages] = useState([]);  
  const [isLoading, setIsLoading] = useState(false);  
  const sessionId = useRef(crypto.randomUUID());  
  
  const sendMessage = async (question) => {  
    setMessages(prev => [...prev, { role: 'user', content: question }]);  
    setIsLoading(true);  
    try {  
      const response = await queryRag(question, auditData, sessionId.current);  
      setMessages(prev => [...prev, {  
        role: 'assistant',  
        content: response.answer,  
        sources: response.sources  
      }]);  
    } finally {  
      setIsLoading(false);  
    }  
  };  
  
  return { messages, isLoading, sendMessage };  
}
```

25. Wire `useRagChat` into the `aiAssistant` feature component.

26. Display source citations below each AI answer as expandable chips.

27. Show a typing animation while the RAG service is generating a response.

Acceptance Criteria: The AI Assistant sends questions to the real RAG service and displays answers with legal citations from DPDP, GDPR, DSA etc.

Feature 6 — PDF Export

Why: Enterprise users need to share reports with legal teams and management. **Implementation Steps:**

28. Install: `npm install @react-pdf/renderer` or use `jspdf` + `html2canvas`

29. Create `src/features/export/ExportButton.jsx` that renders the audit report as a styled PDF:

- Executive Summary section
- Findings table (severity, category, description)
- Compliance framework scores
- Remediation roadmap
- YourCrawl branding footer

30. Add "Export PDF" button to the report header.

31. Optional: Add CSV export for the findings table for data analysis.

Acceptance Criteria: Clicking "Export PDF" downloads a professional-looking PDF of the audit report within 3 seconds.

Feature 7 — Global Error Boundary and Empty States

Implementation Steps:

32. Create `src/components/common/ErrorBoundary.jsx`:

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return (
        <div className="flex flex-col items-center justify-center h-64 text-center p-8">
          <AlertTriangle className="w-12 h-12 text-orange-400 mb-4" />
          <h3>Something went wrong</h3>
          <p className="text-gray-500 mt-2">{this.state.error?.message}</p>
          <button onClick={() => window.location.reload()}>Reload Page</button>
        </div>
      );
    }

    return this.props.children;
  }
}
```

33. Wrap the route tree in `App.jsx` with `<ErrorBoundary>`.

34. Create `src/components/common/EmptyState.jsx` for views with no data (e.g., no audit history yet).

Acceptance Criteria: Any runtime error in a component shows a friendly error message instead of a white screen. Empty views show helpful empty state messages.

Feature 8 — Toast Notifications for Key Actions

Why: `sonner` (toast library) is already installed. Users need confirmation that actions succeeded or failed.

Implementation Steps: Wire `sonner` toast notifications to:

- Audit started: `toast.loading('Audit queued...')`
- Audit complete: `toast.success('Audit complete! 12 findings detected.')`
- Audit failed: `toast.error('Audit failed: Connection timeout')`
- Report saved: `toast.success('Report saved')`

- AI Assistant error: `toast.error('AI Assistant is unavailable')`

Acceptance Criteria: Every major user action results in a clear toast notification at the bottom right of the screen.

Feature 9 — Responsive Design Audit and Fixes

Implementation Steps:

- 40. Test all pages on: 375px (iPhone SE), 768px (iPad), 1280px (Laptop), 1920px (Desktop)
- 41. Priority fixes for mobile:
 - Sidebar should collapse to a hamburger menu on mobile
 - Audit findings table should become cards on mobile
 - Charts should be scrollable horizontally on mobile
 - The URL input form should be full-width on mobile
- 42. Add responsive Tailwind breakpoints to all layout components.

Acceptance Criteria: All pages render correctly and are usable on a 375px mobile viewport.

Delivery Timeline

Week	Milestone
Week 1	Feature 2 (Remove Mock Data + API Layer) + Feature 7 (Error Boundary)
Week 2	Feature 1 (Real-Time Progress UI) + Feature 3 (Skeleton Loading)
Week 3	Feature 4 (Route Discovery Modal) + Feature 5 (AI Assistant Wired)
Week 4	Feature 6 (PDF Export) + Feature 8 (Toasts) + Feature 9 (Responsive)

New Dependencies to Add

```
zustand ^5.x           (global state management)
@react-pdf/renderer ^3.x (PDF export)
axios ^1.x             (better HTTP client than raw fetch)
react-query ^5.x       (data fetching + caching) [optional but recommended]
```

Definition of "Production Ready" for Frontend

- ☐ Zero components importing from `mockData.js`
- ☐ All data-fetching views have loading (skeleton) states
- ☐ All data-fetching views have error states
- ☐ Real-time audit progress bar working with SSE
- ☐ Route discovery modal working
- ☐ AI Assistant wired to real RAG service with citations

- ☐ PDF export working
- ☐ Error boundary wrapping all routes
- ☐ Toast notifications on all key user actions
- ☐ All pages verified on 375px mobile viewport
- ☐ Lighthouse Performance score ≥ 80
- ☐ No console errors in production build